



**CEG2001 SENSORS AND CONTROL
CEG2002 EMBEDDED SYSTEM DESIGN**

JOINT MODULE GROUP PROJECT

Smart moveable Bin

Project Report

Group: JMGP 10

Students names :

ANNIE YEAP HUI EAN (2402873)

BRENDAN TJUNG YEW WANG (2401765)

LAU WENG SUN SAMUEL JOHN (2402388)

LOOI WEI JIE (2402060)

RYAN ANG RUI HENG (2400522)

ZION LIM ZI EN (2401235)

Table of Contents:

1. Abstract	2
2. Introduction	2
3. System Architecture	3
3.2 System Workflow Description	3
1. Start-up workflow	3
2. Base mode	3
3. Follow-Me Mode	3
Camera Detection Mode	4
4. Sensor Selection	4
4.1 ESP32-CAM (AI Thinker) – camera sensor	4
4.2 IR sensors for bin fill detection (digital + analog)	4
4.3 Ultrasonic sensor for person distance detection	5
4.4 IMU for yaw detection	5
4.6 Compare with at least one alternative sensor	5
IR sensors	5
5. Sensor Data Acquisition and Processing	7
6. System Testing and Performance Evaluation	10
7. Summary and Conclusion	11
Summary	11
Conclusion	11
8. Reflection	11
9. Discussion on the use of generative AI and Reflection	13
10. References	13
11. Appendices	15

1. Abstract

This report presents the development of Bin Buddy, an intelligent mobile waste-collection robot designed to support indoor waste-management tasks. Bin Buddy integrates MSP430F5529, ESP32-S3 FireBeetle 2, and ESP32-CAM to achieve three main operating modes: Normal Mode, Follow-Me Mode, and Call-Me Mode. Equipped with ultrasonic sensors, Ai trash detection for trash sorting via the servo-controlled lid, mecanum-drive movement through dual L298N motor drivers, and BLE communication, Bin Buddy autonomously follows a user, responds to wireless commands, detects hand-movement gestures to open its lid, and alerts when its capacity is full. The overall system comprises sensing, actuation, wireless communication, and algorithmic decision processes coordinated through a structured firmware state machine. This report documents the design methodology, system architecture, implementation, testing, and results of Bin Buddy, along with limitations and proposed future improvements.

2. Introduction

Traditional waste management within indoor environments such as schools, offices, and homes remains fully manual. Users are required to walk to a fixed bin location, which may be inconvenient, especially for elderly or mobility-challenged individuals. Modern embedded systems enable automation that improves everyday convenience by digitising routine tasks.

Bin Buddy was developed as an autonomous mobile robot to address these challenges. The primary function of Bin Buddy is to follow its user and automatically sort out the trash given which enhances hygiene by reducing contact with the bin surface. It can also be controlled wirelessly using a joystick or a simple “Call-Me” command.

In addition to the core functionalities, Bin Buddy uses simple IR, ultrasonic sensors and also explores the use of a low-cost camera with an onboard AI model as an extra “smart sensor” in identifying different types of waste (plastic wrappers or paper etc.). This feature aims to enhance automatic sorting thus reducing recycling contamination. This supports the sustainability goal of the project by reducing cross-contamination of waste, improving recycling rates and encouraging users to dispose of items correctly.

Subsequent sections of this report will describe the design, system architecture and sensor choices for this AI camera add-on.

3. System Architecture

3.1 Overall Block Diagram

The system consists of three microcontrollers interconnected:

- MSP430F5529 (main controller)
- .ESP32-S3 FireBeetle 2 (BLE communication + additional processing).
- ESP32-CAM (visual processing)

Sensors (ultrasonic, gesture) provide environment data to MSP430. Actuators (motors and servo) respond to commands. The power system provides regulated connections.

Refer to Figure 1.1 and Figure 1.2

3.2 System Workflow Description

1. Start-up workflow

- All MCUs power on.
- Each subsystem initializes:
 - i. MSP430 initializes motors, GPIO, LEDs
 - ii. ESP32-S3 starts BLE advertising & sensor calibration
 - iii. ESP32-CAM loads camera model
 - iv. Arduino initializes IR sensors & servo
- ESP32-S3 sets bin to **Base Mode (Manual Move)**

2. Base mode

- User send command via BLE → ESP32S3 receives it
- ESP32S3 sends command to MSP430 via I2C
- MSP430 drives the wheel accordingly
- If no command for 30s:
 - i. MSP430 → enters Sleep Mode
 - ii. ESP32S3 → enter light sleep, waiting for BLE input
- ‘W’ command wakes MSP430 and ESP32S3:
 - i. All return to active mode

3. Follow-Me Mode

- User sent ‘2’ (Follow me command)
- ESP32S3 activates Follow-Me logic:
 - i. Reads the front Ultrasonic sensor continuously
 - ii. Detects user distance
- Follow-Me condition
 - i. User must be in front
 - ii. Distance must be valid and stable
- Movement loop:
 - i. If user is 80-150cm → moves forward
 - ii. If user is <10cm → moves backwards
 - iii. If user stop → motor stop
 - iv. If user disappear → motor stop
- ESP32S3 sends direction command to MSP430

4. Camera Detection Mode

- Check IR first
 - i. Read digital and analog IR.
 - ii. If left / right / both full → show “Dry full / E-waste full / Both full”, remain to 90°(detection bed flat) (no camera, no detection).
- Run detection
 - i. Show “Detecting object”.
 - ii. Capture frame, convert to 96×96, run object detection classifier
 - iii. Pick the bounding box with the highest score.
- Map label → side
 - i. “Paper waste” → send item to **left bin** (Dry) if left bin is not full
 - ii. “Broken Controller” → send item to **right bin** (E-waste) if right bin is not full
 - iii. If the mapped bin is full → no sorting, just show “bin full” on the OLED
- Move servo once per object
 - i. OLED shows the detected class and confidence (e.g. “Paper 92%”)
 - ii. Lid rotates towards the selected side so the item drops into that bin:
 - Left bin → lid turns left(25 degree), then returns to centre
 - Right bin → lid turns right(155 degree), then returns to centre
 - iii. System remembers that this item has already been sorted
 - iv. When the object is no longer detected in the camera view, the system “arms” again and waits for the next item

4. Sensor Selection

4.1 ESP32-CAM (AI Thinker) – camera sensor

The ESP32-CAM with the OV2640 is used as the main vision sensor. It is positioned above the lid to capture top-down images of the trash placed on the detection bed. These images are later processed by the Edge Impulse model for classification, but in this section we focus on the camera as a sensor.

Operating Principle and Suitability

The OV2640 is a CMOS camera where light passes through the lens, hits a pixel array, and each pixel converts light into an electrical charge that is amplified and digitised inside the sensor (light → lens → pixels → ADC → digital image), then read by the ESP32 and stored in PSRAM for resizing (e.g. 96×96) and inference. It is small, low-power, supports suitable resolutions that can be downscaled without losing trash shape/texture, and is already integrated on the ESP32-CAM with PSRAM, matching our compact, low-cost smart bin requirements for real-time embedded vision.

4.2 IR sensors for bin fill detection (digital + analog)

Two IR sensors are used together to detect if each bin compartment is “full enough” near the lid: a Flying-Fish digital IR module for the dry-waste side and a Sharp GP2Y0A21YK-type analog IR distance sensor for the e-waste side. Both are mounted near the wall of the bin so that when trash reaches the sensing height, the reflected IR signal changes and the system treats that bin as full.

Operating Principle and Suitability

The **digital IR sensor (Flying-Fish)** emits infrared light and uses an onboard comparator to output a HIGH or LOW signal when an object crosses a fixed threshold. This makes it simple, reliable, and ideal for straightforward “full/not full” detection. The **analogue IR sensor (Sharp GP2Y0A21YK0F)** also emits IR light but outputs a continuous voltage that varies with distance, allowing gradual fill-level measurement. Both sensors are compact, inexpensive, and well-suited for short-range detection in the confined space of a bin. The digital sensor keeps logic simple for binary detection, while the analogue sensor provides finer resolution for applications requiring more detailed feedback.

4.3 Ultrasonic sensor for person distance detection

The ultrasonic sensor measures the distance between the robot and a person, enabling safe spacing and presence detection or trigger specific behaviours when someone approaches. It emits high-frequency sound waves and calculates distance from the echo time to provide reliable real-time feedback.

Operating Principle

The HC-SR04 emits a 40 kHz ultrasonic pulse through its **trigger** pin and measures the returning echo through its **echo** pin. The ESP32 reads the pulse duration and converts it into distance using time-of-flight, allowing the robot to detect nearby people and adjust its movement accordingly

4.4 IMU for yaw detection

The IMU is used in this project to measure the robot’s **yaw angle**, allowing the system to detect and respond to rotational changes. This helps ensure that the mecanum wheels maintain the correct heading during movement, especially when rotating or when external forces cause the robot to drift.

Operating Principle

The IMU contains a 3-axis gyroscope that measures angular velocity around the X, Y, and Z axes. For yaw detection, the robot uses the Z-axis rotation, which represents turning left or right.

The gyroscope outputs angular velocity, which the ESP32 reads through the I2C interface. By integrating this rotational velocity over time, the system calculates the current yaw angle. This value is then used to correct wheel movement, stabilize orientation, and ensure that the robot follows the intended direction even during complex mecanum maneuvers.

4.5 Compare with at least one alternative sensor

Chosen sensor	Alternative	Justification
ESP32-CAM	USB webcam	A standard USB webcam with a laptop or mini PC could also be used, offering higher resolution, better low-light performance, and easy OpenCV integration for data collection and testing.
IR sensors	Weight Sensor	A weight sensor can measure total trash mass but is impractical for mixed, uneven waste, whereas an IR sensor detects fill level more simply and reliably without needing to support the load.
Ultrasonic	Lidar	Lidar provides more accurate distance readings and is less affected by surface texture or angle, allowing the robot to detect the user's position more reliably.
IMU	Magnetometer	A magnetometer can detect heading, but it is easily affected by nearby metal and electrical noise, making the IMU more stable for yaw tracking.

4.6 Key characteristic and limitation

ESP32CAM

Key characteristic	Limitation
Resolution: 2 MP (OV2640)	Lighting Sensitivity: The camera's performance can degrade in poor or inconsistent lighting.
Pixel Format: JPEG, RGB, GRAYSCALE	
Frame size: QVGA (320x240), rescaled to 96x96	Resolution: The 2 MP resolution is adequate for our application, but higher resolutions would be needed for more detailed object recognition.
Power: 3.3v	Field of View: The camera's fixed field of view limits its ability to detect objects from varying distances.

Digital IR Sensor Summary Table

Key Characteristics	Limitations
Digital HIGH/LOW output	Affected by strong ambient light
Range: 5–30 cm	Limited detection range
Operates at 3.3V/5V	Low reflectivity objects may be missed

Digital IR Sensor Summary Table

Key Characteristics	Limitations
Digital HIGH/LOW output	Affected by strong ambient light
Range: 5–30 cm	Limited detection range
Operates at 3.3V/5V	Low reflectivity objects may be missed

Edge Impulse Summary Table

Key Characteristics	Limitations
96x96 inference model	Slow for complex scenes
Supports low-power microcontrollers	Sensitive to background changes
Streamlined training and deployment	–

ESP32 FireBeetle Summary Table

Key Characteristics	Limitations
Bluetooth 4.2 / BLE	Limited indoor range (5–10 m)
Dual-core 240 MHz processor	2.4 GHz interference
Operates at 3.7–5V	Slight power overhead

Ultrasonic Sensor Summary Table

Key Characteristics	Limitations
Range: 2–400 cm	Weak performance on sound-absorbing materials
40 kHz ultrasonic signal	Narrow ~15° detection cone
5V supply	Affected by strong air movement
Not affected by lighting	Blind zone < 2 cm

IMU Summary Table

Key Characteristics	Limitations
3-axis angular velocity sensing	Gyroscope drift over time
Uses Z-axis for yaw	Requires filtering (Kalman/complimentary)
I2C communication	Motor vibration introduces noise
Fast sampling, low power	–

5. Sensor Data Acquisition and Processing

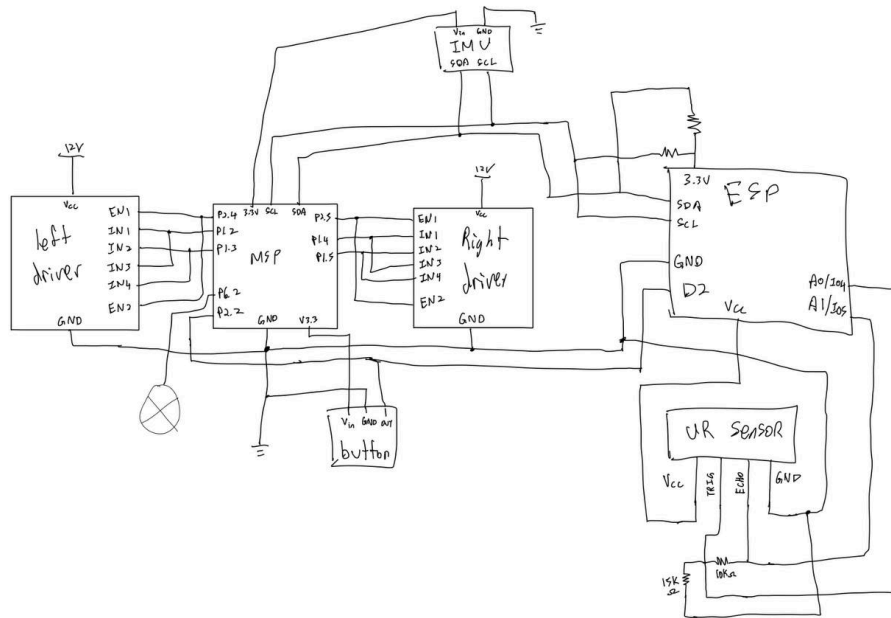


Figure 5.1: Overall schematics

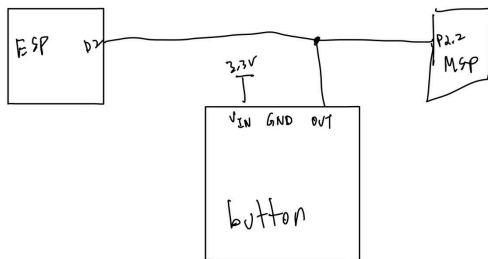


Figure 5.2 :Button connection schematics

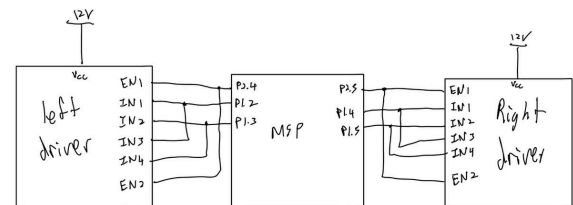


Figure 5.3: wheel driver schematics

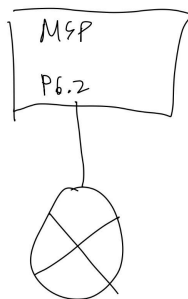


Figure 5.4: LED connection schematics

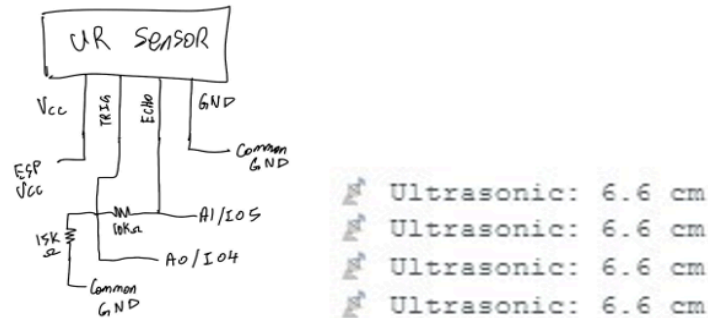


Figure 5.5:UR Sensor configuration and Measured UR sensor

The actual distance between the UR sensor and my testing finger is 6.5cm away. The UR measured 6.6 cm. The stability test for the UR sensor is roughly ± 0.1 cm. This shows that the UR sensor is quite accurate. The average reading from the UR sensor is 6.6cm, error ± 0.1 cm which is acceptable. As the UR sensor is used to detect how close or how far the user is away from the bin during the follow me mode, we have physically measured how far the bin should be before it should move forward and how close the user is to the bin before it has to move back. We have concluded that if the bin is less than 10 cm away from the user it will move back. If the user is between 20cm and 1m away from the bin it will keep moving forward until it senses the user less than 20cm away from the user.

```

Ultrasonic: ---
Ultrasonic: ---

```

Figure 5.6: Sensor Out of range

The ultrasonic sensor is limited to a reliable range of **40–400 cm** but set to maximum distance to 100cm. When the measured distance falls outside this range, or when no stable echo is detected, the system treats the reading as invalid and displays “**Ultrasonic: ---**” as shown in figure 5.6. In this state, the robot does not move, preventing unwanted motion when the user is too far away or when reflections cause unreliable readings. Normal Follow-Me behaviour resumes only when a valid distance is detected again.

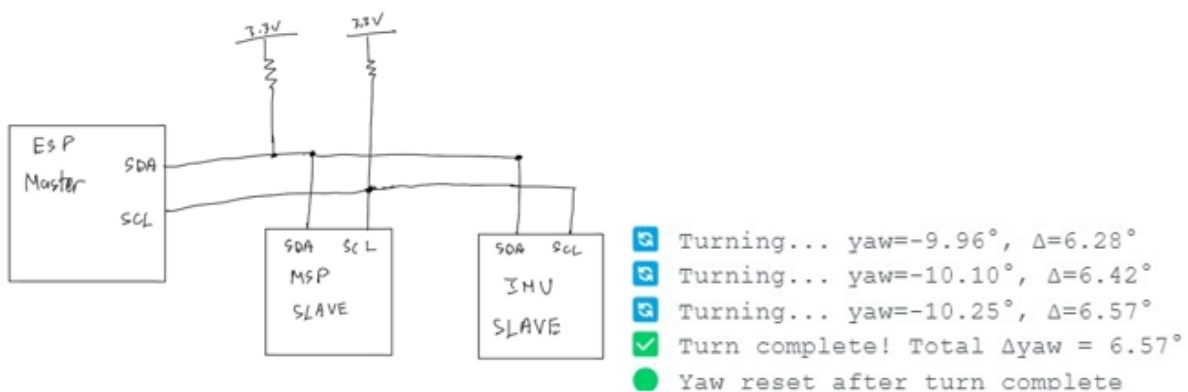


Figure 5.7: I2C connection schematics and IMU reading

In our system, the yaw-angle data is retrieved from the IMU through the I2C interface. The esp32 polls the sensor at regular intervals and retrieves raw angular velocity and orientation values. These values are then processed by the ESP32 to determine when the bin has turned 90 degrees. The IMU output yaw values in degrees. However, we notice that the values do not directly represent the physical rotation of the robot. When we turn the robot 90 degrees on the table we notice that the sensor provides a scaled change in orientation rather than the actual orientation. The sensor showed an approximate 10.25 degree turn when the physical turn is at 90 degree. With this information, we then used it to establish a baseline yaw value as 0 every time we had to turn the bin. We will then process the reading as change of yaw = current yaw - yaw start. With the predefined value we found when turning the robot on the table we then set it to identify the yaw change to read it as it had completed the 90 degree turn.

To filter out the noise caused by the vibration, we lower the speed of the turn when the bin is not loaded on the platform. With the bin loaded, with the weight pressing down reducing the vibration, we could increase the speed as the weight added on by the bin is enough to neutralize the vibration to allow the IMU to provide us with a relatively accurate reading.

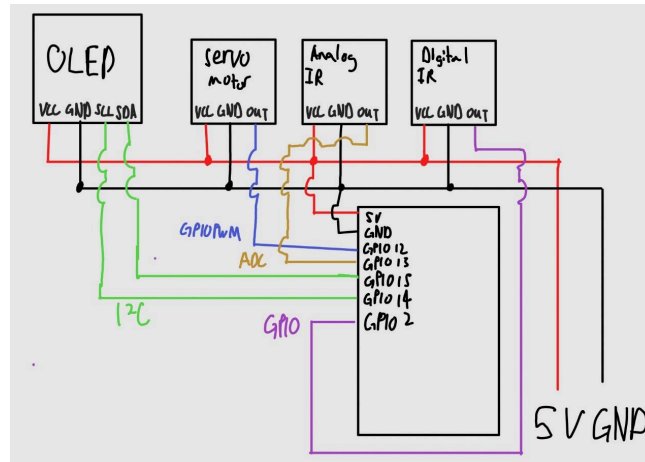


Figure 5.8: Circuit connection for object detection

The object detection sensor consists of esp32-cam + Edge Impulse model. The model was first trained to an F1 score of about 92.8% on the validation set, then retrained with more data until the confusion matrix and test metrics reached 100% for Background, Paper Waste, and Broken Controller. For the analog IR distance sensor, we chose the “bin full” threshold by checking ADC readings at known fill levels and picking a value that indicates when trash reaches the sensing level. Image pixels are resized to 96×96 before inference, and raw ADC / digital readings are converted into simple flags such as “Dry full”, “E-waste full”, or “bin OK”.

Refer to Appendix [Figure 5.9](#) ,[Figure 5.10](#) and [Figure 5.11](#) for image labelling examples, model test output and model metrics

Outputs are displayed as OLED text and serial logs. The OLED displays messages like “Paper 69%” or “Controller 85%” for detections, and “Dry full”, “E-waste full”, or “Both full” when the IR sensors trigger. The Serial Monitor prints the IR states and the bounding-box results (label, confidence, coordinates), which can be turned into tables or simple plots to see how the system behaves over time.



Figure 5.11: *F1 before adjustment*

Figure 5.11: *F1 after adjustment*

Another challenge we face is from the early model, even with a 92.8% F1 score tested online, it was not that accurate in our actual test. Some time papers were still detected as controllers under different lighting or positions. Further testing made us realise that the dataset did not fully cover our real scenarios. We collected more pictures and retrained until both the Edge Impulse metrics and bench tests showed consistent sorting with a F1 score of 100%.

6. System Testing and Performance Evaluation

```

Turning... yaw=5.78°, Δ=6.32°
Ultrasonic: ---
Turning... yaw=5.85°, Δ=6.39°
Ultrasonic: ---
Turning... yaw=5.94°, Δ=6.47°
Ultrasonic: ---
Turning... yaw=6.00°, Δ=6.53°
Turn complete! Total Δyaw = 6.53°
Yaw reset after turn complete

```

Figure 6.1: Image for yaw

We did a test on the IMU yaw-angle reading to evaluate the accuracy and reliability of the IMU’s yaw-angle reading during a 90 degree turning motion. This test is essential as it validates the IMU ability to complete the turn accurately under different conditions. We connect the IMU to the bin via the I2C interface to the ESP32. From the ESP32 a turn command was sent to perform a 90 degree turn. We tested the IMU in 2 environments: tabletop surface and the ground surface. For each test, we recorded the change of yaw recorded by the IMU and expected physical rotation. After analysing the IMU behaviour, a calibrated change of yaw threshold was implemented and retested.

When testing the robot's physical 90 degree rotation on the table, the IMU only recorded ~10.25 degree yaw-angle and the change in yaw is ~7 degree. This shows that the IMU outputs are scaled orientation values instead of the physical true degree. Although the reading from the IMU is inaccurate in magnitude,

the reading given was consistent and repeatable across multiple trials. With this conclusion, we have identified a reliable threshold for a 90 degree turn.

After getting the threshold from the table testing, we applied the test on the ground. We first keep all the settings to be the same as when it was on the table to identify how it will react. We found out that it is unable to get an accurate reading on the ground to stop when the 90 degree is complete. After we found that that vibration could cause the IMU to have misreading, we lowered the speed of the motor; it is able to perform the 90 degree turn accurately multiple times.

With this test, we have found out that the IMU does not provide us with the physically accurate yaw values. But the change of yaw reading was predictable and consistent, which allowed an accurate 90 degree turn.



Figure 6.2.1: Paper accuracy"



Figure 6.2.2: "Controller accuracy"



Figure 6.2.3: "Dry Full"



Figure 6.2.4: "E-waste full"



Figure 6.2.5: "Both Full"

We tested the vision part by placing one object at a time on the green lid under the ESP32-CAM, first the "controller" object and then crumpled paper. The OLED reported the detected class and confidence (for example "Paper 69%", "Controller 85%") and the lid rotated to the correct side in our final trials, matching the retrained Edge Impulse model that now targets close to 100% accuracy. *See Appendix Figure 6.3 for successful output in the serial monitor console*

To test the bin-full logic, we did not actually fill the bins; instead, we moved a hand to the sensing height of each IR sensor to mimic a full compartment. The OLED then switched to "Dry full", "E-waste full" or "Both full", and the servo stayed at the centre, so no sorting was triggered. This meets the design goal of

blocking, sorting and warning the user when a bin is considered full, with remaining limitations mainly from sudden lighting changes or objects very different from the training images.

To improve energy efficiency, the system implements an automatic and manual low-power mode across both microcontrollers. Auto-sleep is triggered when no movement or BLE command is received for 30 seconds, while manual sleep is activated when the user sends the 'Z' command. In both cases, the MSP430 enters LPM0, turning off the CPU and motor PWM outputs while keeping the I2C module and interrupt system active so it can still receive a wake command. The ESP32-S3 switches to a BLE-safe idle mode, reducing CPU frequency and Wi-Fi power while leaving the Bluetooth radio fully active to maintain responsiveness. Before entering sleep, the motors are forcibly stopped, the MSP430 status LED changes to yellow, and all regular sensor polling (IMU, ultrasonic, and IR readings) is paused.

The system supports multiple wake-up sources to ensure reliability. Sending the 'W' command over BLE immediately wakes the ESP32-S3, which then notifies the MSP430 through I2C to exit LPM0. A physical button connected to P2.2 on the MSP430 also triggers a PORT2 interrupt that wakes the controller even if BLE is not available. Additionally, reconnecting the BLE device causes the ESP32-S3 to resume full operation and send a wake command to the MSP430 automatically. During low-power mode, the only components that remain active are the MSP430's I2C and interrupt hardware, and the ESP32's BLE radio and GPIO wake circuitry. All motors, sensors, and high-frequency processing are switched off, ensuring minimal power consumption while preserving the ability to wake immediately on command or physical interaction.

7. Summary and Conclusion

Summary

This project developed a multi-functional autonomous robot capable of navigation, waste sorting, and human interaction. The system integrates the FireBeetle ESP32 for Bluetooth control, an IMU for yaw detection, ultrasonic sensors for human-distance tracking, and IR sensors for bin-level monitoring, all working with a mecanum wheel drive for precise omnidirectional movement.

The robot operates in three modes: Follow-Me Mode, which maintains a safe distance using ultrasonic feedback and IMU stabilization; Manual Mode, which provides direct mobile Bluetooth control; and Low-Power Mode, which limits sensing and movement to extend battery life.

Beyond mobility, the robot includes an automated waste-sorting mechanism that differentiates between paper and e-waste, along with a fill-level detection system for both bins. Together, these functions combine real-time sensing, user control, and autonomous behavior into a compact and reliable embedded system.

Conclusion

The completed robot successfully meets the original project objectives by combining mobility, environmental awareness, and user interaction into a single platform. Through the use of mecanum wheels and sensor-supported navigation, the robot is able to operate both autonomously and under manual control, offering flexibility for different scenarios. The Follow-Me mode showcases the system's ability to support human-robot interaction, while the low-power mode highlights thoughtful energy management in embedded design.

The inclusion of an e-waste and paper sorting mechanism further expands the robot's practical value by contributing to waste management and sustainability efforts. These capabilities, supported by reliable sensor integration and a stable control architecture, demonstrate the effectiveness of the chosen hardware components and communication methods.

This project ultimately demonstrates how embedded technologies, when combined with intelligent sensing and motion systems, can produce a versatile and functional robotic platform. The outcomes reflect a strong foundation for future improvements such as enhanced object recognition, wireless data logging, cloud connectivity, or more advanced autonomous navigation.

8. Reflection

Rein: This project has been a real learning experience, doing model training was quite frustrating because the Edge Impulse scores looked good from the onsite model testing but real-world detection was different, and i had to keep collecting data, tuning and retraining before it finally work as intended, but that process had helped me to understand how AI detection really works in practice. I picked up a useful skill in a growing AI space, and I now see the camera + model as a “smart sensor” that fits the CEG2001 idea of sensor and controller, while the using ESP32-CAM, OLED, IR sensors and servo integration fits the CEG2002 focus on embedded systems and interfacing. If we had more time and budget, I would switch to a Raspberry Pi running a YOLO model instead of FOMO, because YOLO is designed for full object detection with higher accuracy, better handling of multiple objects and more mature, optimised implementations, which would likely give faster and more robust detection than our current lightweight FOMO setup.

Zion: Working on the smart bin’s movement, sensing, and control system was both challenging and rewarding. My work involved designing, assembling and programming the mecanum wheel drive system, integrating multiple sensors, implementing control logic, and ensuring the mechanical and electrical components worked together reliably. Troubleshooting wheel movement, correcting motor driver wiring, and balancing PWM outputs is important. I worked closely with my peers on the project and implemented the low-power functionality, which strengthened my understanding of energy-efficient design and event-driven operation. Throughout the process, I faced issues ranging from reversed motor polarities to inconsistent sensor feedback, but each problem pushed me to dig deeper, test thoroughly, and refine both the hardware and software. Overall, this project strengthened my practical understanding of embedded systems, reinforced the importance of debugging and systematic testing, and helped me connect theory to real-world implementation. With more time, I would improve its efficiency on how it can be better controlled with higher motion accuracy using motor encoders and further enhance its features and have greater reliability.

Brendan: This project came with several challenges, especially when we had to figure out which components to use and whether they could work together reliably. We also had to search online for many of our solutions, and finding MSP430 tutorials or examples that matched what we needed was quite difficult. I tested several of the sensors during integration, and through this I learned how easily readings could change because of noise, wiring, or even small adjustments in placement. One important lesson I learned was that if something doesn’t work after trying for quite some time, it’s better to move on, try another approach, or replace the component instead of getting stuck. These experiences showed me how much patience, problem-solving, and flexibility are needed in real embedded system development. The project also helped me better understand concepts from Embedded Systems and Sensors and Control by seeing how they applied directly to hardware. If we had more time, I would like to be more involved in the coding and tuning so I can build more confidence in integrating sensors and solving embedded system issues.

Annie: This project gave me hands-on experience applying what we learned in class to a real system. It was interesting to see how our ideas became a working prototype, and it helped me understand how hardware and code come together in an actual embedded application. A major challenge for me was

figuring out how to properly connect the different sensors to the MSP430 without damaging anything. I was initially unsure about the wiring and voltage requirements. To overcome this, I researched each sensor in detail and tested them in small sections before combining everything. This step-by-step approach helped me build confidence and understand how the sensors functioned. This project strengthened my understanding of both modules. It allowed me to apply theoretical concepts from lectures—such as microcontroller programming, sensor interfacing, and embedded system design—to something practical. With additional time or better resources, I would improve the organisation of the wiring, enhance stability, and refine the system to make it more reliable. I would also explore better testing tools to help identify issues faster.

Samuel: This project was interesting and challenging as it was the first time we worked on a combined module project. I have gained valuable exposure on how different sensors, microcontrollers, and communication methods come together in a real embedded system outside of the sensors, actuators and microcontrollers taught in our classes. I learned the importance of careful wiring, testing, and understanding each component's behaviour after damaging our ESP32 used for the Trash detection by incorrectly wiring the microcontroller. I overcame my challenges mainly through research, working closely with my teammates and learning from my mistakes. The project strengthened my understanding of both modules CEG2001 helped me relate sensor principles to real applications, while CEG2002 showed how embedded systems are programmed and integrated. If I have been given more time and resources, I would like to improve the follow-me function by integrating another Camera module and AI to track the user more accurately as our current solution using ultrasonic sensors is very inefficient.

Ryan: This project was one of the more challenging yet rewarding experiences. Throughout this project, it has tested my ability to be able to do extensive debugging, research and hands-on testing. One of the earliest challenges I encountered was establishing reliable communication between 2 MCUs. Initially I tried to implement UART communication. However, after intensive testing, I realized that asynchronous nature introduces timing inconsistencies that have made our system unreliable for our system's real time requirement. I then redesign the communication layer using I2C instead. With the synchronous clock, I2C provided a more consistent and stable communication for our bin. A large portion of my work also involved understanding how different sensors behaved under real-world conditions. Through repeated testing, I learned how easily readings could be affected by factors such as vibration, noise, lighting conditions, and mechanical load. This helped me better appreciate the importance of calibration, filtering, and robust data-processing logic—especially when working with IMUs and distance sensors. Managing these technical challenges built my confidence in both troubleshooting and systematic problem-solving. If I had more time, I would perfect our follow-me mode by incorporating the Ultrawideband(UWB). This will help us to be able to pin-point the location of the user inside the room. This will help with the follow-me and find-me mode.

9. Discussion on the use of generative AI and Reflection

In this project, we used ChatGPT to help us understand how to connect and use our sensors, fix code errors, and troubleshoot problems when things did not work as expected. Since it was sometimes hard to find MSP430 examples online, AI made it easier for us to learn quickly by giving explanations, sample code, and different ways to solve an issue. Some prompts worked better than others, for example, asking “Why is my sensor reading unstable?” gave helpful reasons related to noise or wiring, and asking “Help me debug this code” helped us find mistakes in our logic. Prompts about integrating sensors were useful too, even though we still had to test and adjust them ourselves. Using generative AI had clear advantages, such as saving time, giving fast answers, and helping us understand new concepts, but it also had limitations because not every suggestion worked perfectly on real hardware. Overall, AI was a helpful tool that supported our project, but we still needed hands-on testing and what we learned in CEG2001 and CEG2002 to make everything work properly.

10. References

- Cite all the sources and references you used in your report, following a consistent citation style

https://wiki.dfrobot.com/SKU_DFR0975_FireBeetle_2_Board_ESP32_S3

<https://www.ti.com/product/MSP430F5529>

<https://components101.com/modules/esp32-cam-camera-module>

<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

<https://www.handsontec.com/dataspecs/module/L298N%20Motor%20Driver.pdf>

<https://www.datasheethub.com/ssd1306-128x64-mono-0-96-inch-i2c-oled-display/>

11. Appendices

- Include any supplementary material, such as additional data, or charts/graphs

Camera detection block diagram

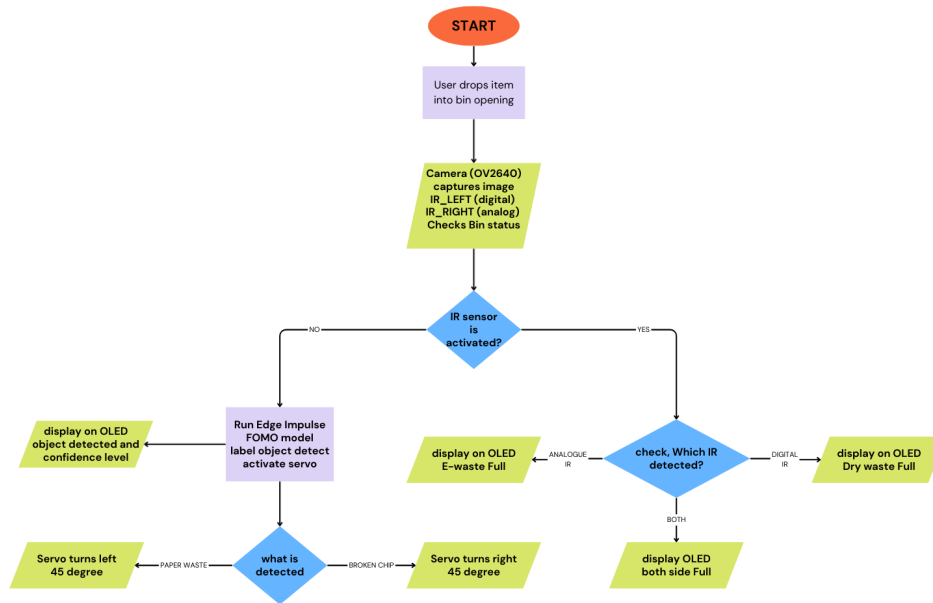


Figure 1.1

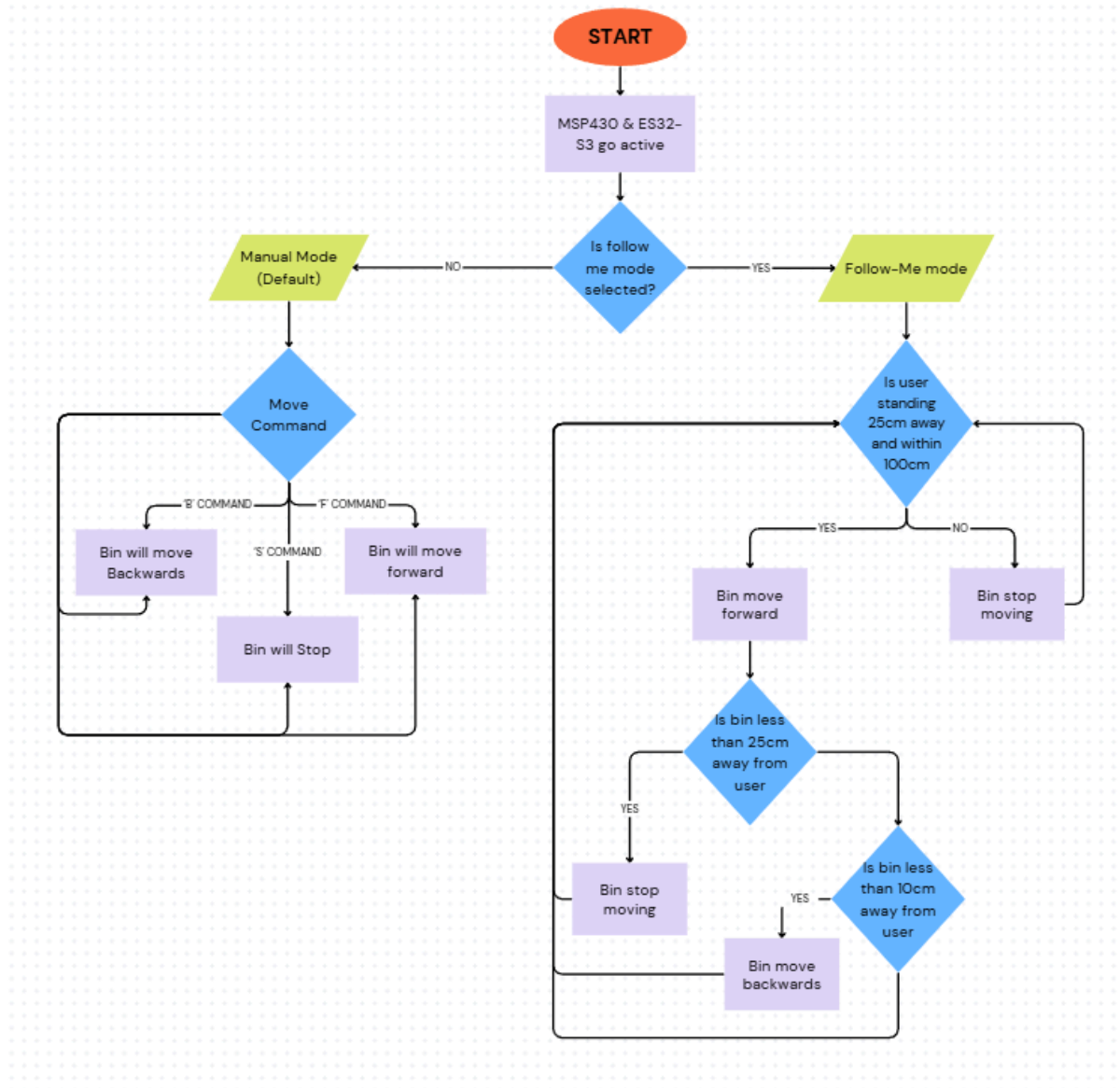


Figure 1.2

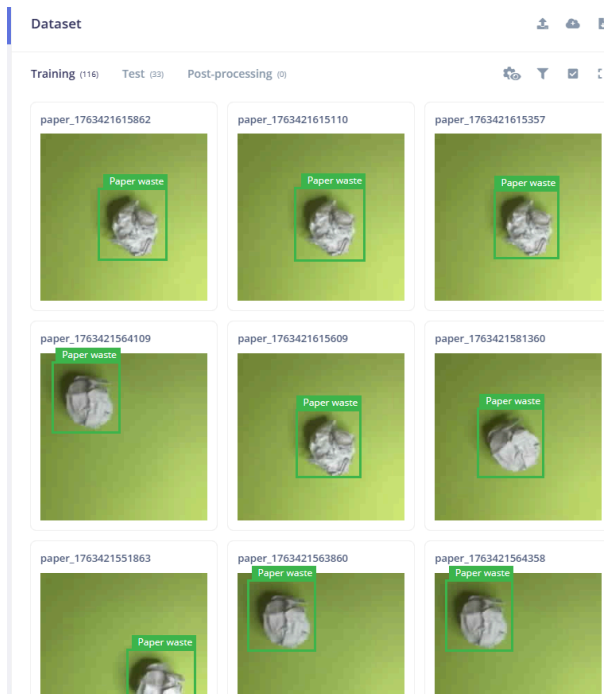


Figure 5.9: image labelling examples

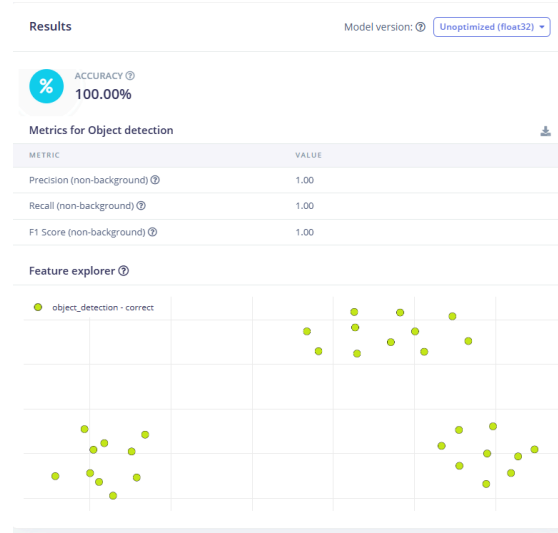


Figure 5.10: model test output

Confusion matrix (validation set)

	BACKGROUND	BROKEN CONTROLLER	PAPER WASTE
BACKGROUND	100%	0%	0%
BROKEN CONTROLLER	0%	100%	0%
PAPER WASTE	0%	0%	100%
F1 SCORE	1.00	1.00	1.00

Figure 5.11: model metrics

```

22:28:36.992 -> IR L=1 R=1
22:28:37.987 -> Predictions (DSP: 8 ms., Classification: 811 ms., Anomaly: 0 ms.):
22:28:37.987 -> Object detection bounding boxes:
22:28:37.987 ->   Broken Controller (0.500000) [ x: 64, y: 40, width: 8, height: 8 ]
22:28:37.987 -> IR L=1 R=1
22:28:38.992 -> Predictions (DSP: 8 ms., Classification: 811 ms., Anomaly: 0 ms.):
22:28:38.992 -> Object detection bounding boxes:
22:28:38.992 ->   Broken Controller (0.527344) [ x: 64, y: 40, width: 8, height: 8 ]
22:28:38.992 -> IR L=1 R=1
22:28:39.991 -> Predictions (DSP: 8 ms., Classification: 811 ms., Anomaly: 0 ms.):
22:28:39.991 -> Object detection bounding boxes:
22:28:39.991 -> IR L=1 R=1

```

Figure 6.3: serial monitor readings

Youtube Video: https://youtu.be/5REJ_TuH1iE